Project Report
Annalin Dequental

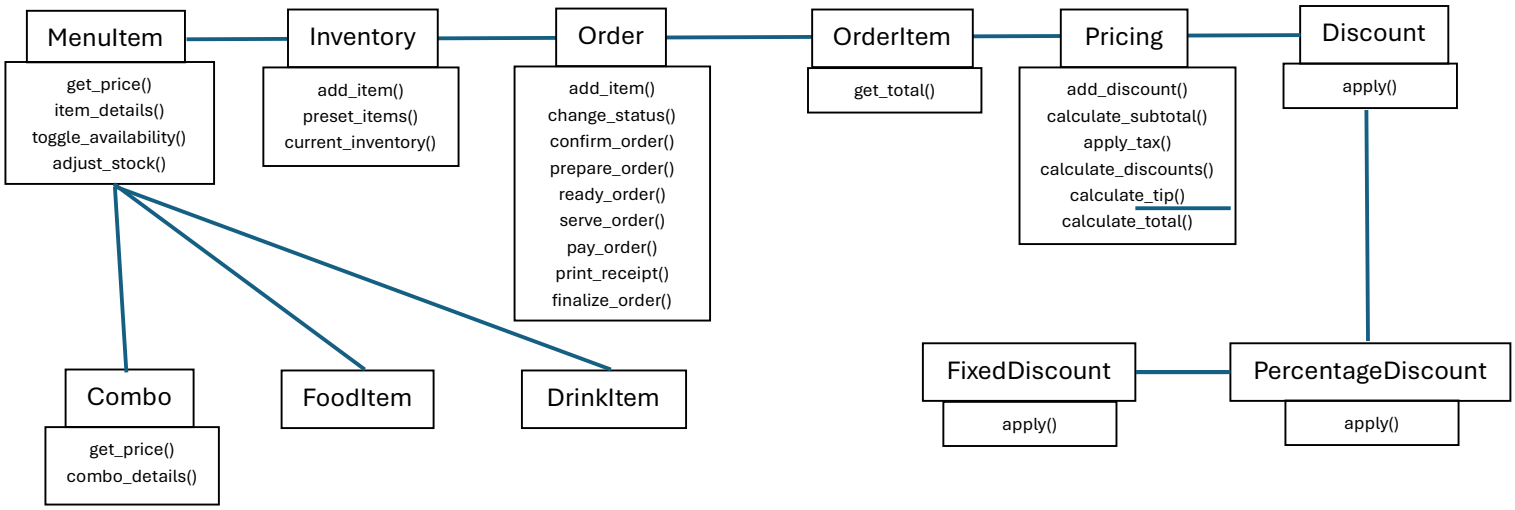
Description:

This project designs a restaurant ordering system in which the standard functions of a restaurant can be performed. The system includes the keeping of an inventory and menu, while being able to track orders and calculate prices. Overall, this project is based on Object Oriented Programming and principles such as Inheritance, Encapsulation, etc. So, the main elements used to perform each of functions of the system are through the methods of Classes and Objects. This program involves several classes although the main functions of the system are carried out through the Inventory, MenuItem, Order, and Pricing classes. Then, processes such as updating stock, getting the price of an item, and creating orders can be completed.

Organization and Overview:

This program contains 4 main modules: inventory, menu, pricing, and orders. Each of which contains various classes and subclasses that interact to perform certain actions. In the inventory module, by creating an object from the Inventory class and passing the correct arguments, details about the restaurant inventory can be managed by the class methods. Similarly, in the menu module, through the MenuItem class and its subclasses, details of menu items can be accessed or changed. For example, by creating an object from MenuItem and passing values for id, name, price, category, available, and stock, then calling the "toggle_availability" method, the availability of the item will be changed. These same steps and concepts can be applied to the Order and Pricing classes to complete other actions.

Class Diagram:



Brief Summary of Developed Classes:

Inventory: Adds items to the menu, keeping the items that are preset to the menu, and accesses the current inventory. Items are categorized as food or drinks and are tracked using specific ids. Class attributes: none; instance attributes: menu_items and next_id.

MenuItem: Allows for new menu item objects to be created. Can return the price or details of an item and the availability or stock of MenuItem objects can be altered. Class attributes: none; Instance attributes: id, name, price, available, and stock.

FoodItem: This class is a subclass of the MenuItem class and allows item objects to be created in a similar manner, although all objects created by this class have their category attribute set to "food". Class attributes: none; Instance attributes: id, name, price, "food", and available.

DrinkItem: This class is a subclass of the MenuItem class and allows item objects to be created in a similar manner, although all objects created by this class have their category attribute set to "drink". Class attributes: none; Instance attributes: id, name, price, "drink", and available.

Combo: This class is another subclass of MenuItem and can create combo menu items. It is also capable of returning the price of a combo and the details of a combo, such as the items within it. No class attributes; Instance attributes: id, name, items, available, and discount.

OrderItem: Can create objects of certain menu items to be ordered by the user and return the total price of the order based on how many of an item is ordered. No class attributes; Instance attributes: item and quantity.

Order: This class runs the ordering process. Its methods add items to the order unless it has already been created, change order status, confirm, prepare, ready, serve, and pay for orders. Also, it contains a method to print the receipt of an order. Class attributes: status_descriptions; Instance attributes: items and status.

Discount: Sets the default discount amount to zero. No class attributes or instance attributes.

PercentageDiscount: This class is a subclass of the Discount class and can create discounts of a percentage then apply them. No class attributes; Instance attributes: percent

FixedDiscount: This class is another subclass of the Discount class and can create discounts of a fixed amount then apply them. No class attributes; Instance attributes: amount

Pricing: This class is responsible for applying discounts, taxes, and tips on an order then calculating its subtotal and final price. Class attributes: food_tax and drink_tax; Instance attributes: tip_percent and discounts

Sample Scenario:

For the sample scenario, at least 3 items need to be created from the restaurant ordering system: 2 foods, 1 drink and 1 combo that has a 10% discount. Then, the stock of these items is to be adjusted to 10, 8, 20, and 5, respectively. Next, an order should be created with multiple items of various quantities, including the combo. Additionally, a percentage discount, fixed discount, set taxes, and tip should be applied to the order. Lastly, the order should display the order status process then print a receipt and the stock decremented.

Pricing = (subtotal – total discounts) + tax + tip

SCENARIO INPUT:

→ Creates Items (discount is already preset to 10% in the Combo class)

```
food1 = FoodItem(1, "Burger", 8.99)
food2 = FoodItem(2, "Pizza", 10.99)
drink1 = DrinkItem(3, "Sparkling Water", 3.99)
combo1 = Combo(4, "Burger Combo", [food1, drink1])
```

→ Sets Stock

```
food1.adjust_stock(10)
food2.adjust_stock(8)
drink1.adjust_stock(20)
combo1.adjust_stock(5)
```

→ Creates Order

```
order = Order()
order.add_item(food1, 2)
order.add_item(food2, 1)
order.add_item(drink1, 3)
order.add_item(combo1, 1)
```

→ Applications for Discounts, Taxes, and Tips

```
pricing = Pricing(tip_percent=0.15)
pricing.add_discount(PercentageDiscount(0.10))
pricing.add_discount(FixedDiscount(5))
```

→ Status Changes

```
order.confirm_order()
order.prepare_order()
order.ready_order()
order.serve_order()
order.pay_order()
```

→ Receipt and See Decrements

```python
order.print_receipt(pricing)
order.finalize_order()

print("\n---- Updated Stock ----")
for item in [food1, food2, drink1, combo1]:
    print(f"{item.name}: {item.stock}")
```

OUTPUT:

```
Order confirmed
Order preparing
Order ready
Order served
Order paid
```

```
-------- RECEIPT --------
Burger x2 = $17.98
Pizza x1 = $10.99
Sparkling Water x3 = $11.97
Burger Combo x1 = $11.68

------------------------
Subtotal: $52.62
Discount: $10.26
Tax: $2.70
Tip: $6.35
.......................

Total: $51.41
------------------------
```

In the scenario, various menu objects are created, each fulfilling the necessary arguments. Prior to creating and fulfilling the quantity of the items when called are:

```
---------- Current Stock ----------
Burger: 10
Pizza: 8
Sparkling Water: 20
Burger Combo: 5
```

After fulfilling the order, the same items have the quantities:

```
---- Updated Stock ----
Burger: 8
Pizza: 7
Sparkling Water: 17
Burger Combo: 4
```

Which verifies that items can be successfully added and removed from the inventory after orders.

Conclusion:

When working on this project I found myself enjoying the concept, although I did face some challenges throughout the process. These main challenges being that I was unsure of how to format my program since I had not used multiple modules before and making sure there were no gaps in the functioning of the restaurant system. However, after considering the main elements of this project, I decided to organize my program based upon the main elements of a restaurant: the inventory, menu, pricing/payments, and orders. Once I was able to do that, I found it easy to work out what classes and methods needed to be created to fulfill those categories. Moreover, the basis of this project in Object Oriented Programming (OOP) was beneficial as its principles fit into this format well. Nonetheless, I feel that despite the challenges I was able to achieve the overall goal for this project and was allowed to consider the implementation and usage of OOP for various purposes.

Limitations:

- There are no limits on the stocking of inventory items
- There are no limits on discounts
- Orders must be finalized manually, so the stock will not decrement unless finalizations are called with the completion of any order

Improvements:

- An exception can be added to the confirm_order method in the order class to prevent an order from being submitted if the stock is less than the amount of each item in the order
- An exception can also be made to modify the Discount class so discounts cannot make the subtotal negative. Instead, if the discount surpasses the order subtotal, the price is set to $0 before tip and tax
- The details of the finalize_order method can be added to the confirm_order method, so the stock is adjusted as soon as an order is accepted.